

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5112 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

C04: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

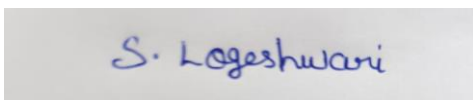
TOTAL MARKS: 25

Annexure A Coding Challenge

Code of Conduct:

I Logeshwari S (192565040)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "S. Logeshwari".

QUESTION 1

Question:

Write a Python program to simulate Secure Email Transmission using S/MIME, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.

Answer

Aim:

To simulate secure email transmission using the S/MIME concept by protecting an email message with encryption and a digital signature.

Algorithm:

1. Generate an RSA private key and corresponding public key for the sender.
2. Generate a symmetric session key for encrypting the email message.
3. Encrypt the message using the symmetric session key.
4. Generate a digital signature for the original message using the sender's private key.
5. Verify the signature using the sender's public key.
6. Decrypt the encrypted message using the session key.
7. Display the verification result and decrypted message.

Python Code

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.fernet import Fernet

# Generate sender's RSA key pair
sender_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
sender_public_key = sender_private_key.public_key()

# Generate a symmetric session key
session_key = Fernet.generate_key()
cipher = Fernet(session_key)

message = input("Enter email message: ").encode()

# Encrypt the email message
encrypted_message = cipher.encrypt(message)

# Generate digital signature
```

```

signature = sender_private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nEncrypted message:", encrypted_message.decode())
print("Digital signature generated successfully.")

# Verify digital signature
try:
    sender_public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    print("Digital signature verification: SUCCESS")
    print("Authentication and integrity verified.")
except Exception:
    print("Digital signature verification: FAILED")

# Decrypt the email message
decrypted_message = cipher.decrypt(encrypted_message)
print("Decrypted message:", decrypted_message.decode())

```

Input:

```
Enter email message: Meeting is scheduled at 10 AM
```

Output:

```

Encrypted message: <ciphertext generated during execution>
Digital signature generated successfully.
Digital signature verification: SUCCESS
Authentication and integrity verified.
Decrypted message: Meeting is scheduled at 10 AM

```

Analysis

Confidentiality is achieved because the email content is encrypted using a symmetric session key. Even if an unauthorized person obtains the encrypted message, the original message cannot be directly read without the key. Authentication and integrity are achieved through the digital signature. The sender signs the original message using the private key, and the receiver verifies the signature using the sender's public key. A successful verification indicates that the message came from the expected sender and was not modified after signing. In a complete S/MIME deployment, certificates issued by a trusted Certificate Authority are also used to bind identities to public keys.

Result:

Thus, the program successfully demonstrates the main S/MIME security objectives: confidentiality, authentication, integrity, and support for non-repudiation.

QUESTION 2

Question:

Develop a Python application to implement PGP-based file encryption and decryption, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.

Answer

Aim:

To implement a simplified PGP-style hybrid encryption process for a file, demonstrating key generation, secure session-key protection, file encryption, and authentication-related processing.

Algorithm:

1. Generate the receiver's RSA public/private key pair.
2. Generate a random symmetric session key.
3. Encrypt the file contents using the symmetric session key.
4. Encrypt the session key using the receiver's public key.
5. Store the protected session key and encrypted file data.
6. Use the receiver's private key to recover the session key.
7. Decrypt the file using the recovered session key.
8. Compare the recovered file with the original file to confirm successful recovery.

Python Code

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.fernet import Fernet
from pathlib import Path

# Generate receiver's public/private key pair
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()

filename = input("Enter file name to encrypt: ")
path = Path(filename)

# Create a sample file if it does not exist
if not path.exists():
```

```

    path.write_text(input("File not found. Enter file content: "))

data = path.read_bytes()

# Generate a symmetric session key
session_key = Fernet.generate_key()
cipher = Fernet(session_key)

# Encrypt file data
encrypted_data = cipher.encrypt(data)

# Protect the session key with receiver's public key
encrypted_key = public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

encrypted_file = path.with_suffix(path.suffix + ".pgp")
encrypted_file.write_bytes(encrypted_key + b"\n" + encrypted_data)

print("File encrypted successfully.")

# Decrypt the protected session key
stored = encrypted_file.read_bytes()
encrypted_key, encrypted_data = stored.split(b"\n", 1)

recovered_key = private_key.decrypt(
    encrypted_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt the file
decrypted_data = Fernet(recovered_key).decrypt(encrypted_data)

decrypted_file = Path("decrypted_" + path.name)
decrypted_file.write_bytes(decrypted_data)

print("File decrypted successfully.")
print("Original and decrypted file content match:",
      decrypted_data == data)

```

Input:

```
Enter file name to encrypt: message.txt
If message.txt does not exist, enter file content: Confidential PGP message
```

Output:

```
File encrypted successfully.
File decrypted successfully.
Original and decrypted file content match: True
```

Analysis

PGP uses a hybrid approach because symmetric encryption is efficient for large amounts of data, while public-key cryptography is suitable for protecting the small session key. The public key can be distributed to the sender, while the corresponding private key is kept secret by the receiver. This combination provides confidentiality with good performance. PGP systems can additionally use digital signatures to provide authentication, integrity, and non-repudiation.

Result:

Thus, the program demonstrates secure file encryption and decryption using a PGP-style hybrid mechanism and shows why combining symmetric and asymmetric cryptography is efficient.

QUESTION 3

Question:

Implement a Python program to simulate IPSec Tunnel Mode, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.

Answer

Aim:

To simulate IPSec Tunnel Mode by representing an IP packet as a complete unit and encrypting the complete original packet using symmetric encryption.

Algorithm:

1. Read the source IP address, destination IP address, and packet data.
2. Construct the complete original IP packet.
3. Generate a symmetric encryption key.
4. Encrypt the complete original packet.
5. Display the encrypted tunnel packet.
6. Decrypt the packet at the receiving endpoint.
7. Display the recovered original packet.
8. Explain the difference between Tunnel Mode and Transport Mode.

Python Code

```
from cryptography.fernet import Fernet

# Generate a symmetric key
key = Fernet.generate_key()
cipher = Fernet(key)

source_ip = input("Enter source IP: ")
destination_ip = input("Enter destination IP: ")
data = input("Enter packet data: ")

# Represent the complete original IP packet
packet = (
    f"Source IP: {source_ip} | "
    f"Destination IP: {destination_ip} | "
    f"Data: {data}"
)
```



```
print("\nOriginal IP Packet:")
print(packet)

# IPSec Tunnel Mode simulation:
# encrypt the complete original packet
encrypted_packet = cipher.encrypt(packet.encode())

print("\nEncrypted Tunnel Packet:")
print(encrypted_packet.decode())

# Receiver decrypts the packet
decrypted_packet = cipher.decrypt(encrypted_packet).decode()

print("\nDecrypted Original Packet:")
print(decrypted_packet)
```

Input:

```
Enter source IP: 192.168.1.10
Enter destination IP: 10.0.0.5
Enter packet data: Hello Server
```

Output:

```
Original IP Packet:
Source IP: 192.168.1.10 | Destination IP: 10.0.0.5 | Data: Hello Server

Encrypted Tunnel Packet:
<encrypted ciphertext>

Decrypted Original Packet:
Source IP: 192.168.1.10 | Destination IP: 10.0.0.5 | Data: Hello Server
```

Analysis

In Tunnel Mode, the entire original IP packet is protected and encapsulated inside a new IP packet. The original IP header and payload are therefore protected from direct observation across the tunnel. Tunnel Mode is commonly associated with VPN gateway-to-gateway or gateway-to-host communication.

In Transport Mode, the original IP header remains in place and mainly the IP payload is protected. Therefore, Transport Mode has lower overhead and is commonly used for host-to-host protection.

Result:

The program successfully simulates the encryption and recovery of a complete IP packet and demonstrates the security advantage of IPSec Tunnel Mode for VPN-style communication.

QUESTION 4

Question:

Write a Python script to implement SSL/TLS handshake simulation, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.

Answer

Aim:

To simulate the main stages of an SSL/TLS handshake, including client/server hello messages, certificate/public-key exchange, session-key establishment, and encrypted application-data transfer.

Algorithm:

1. Generate the server's public/private key pair.
2. Simulate Client Hello and Server Hello messages.
3. Simulate certificate/public-key exchange.
4. Generate a symmetric session key at the client.
5. Protect the session key using the server's public key.
6. Recover the session key at the server using the private key.
7. Encrypt application data using the established session key.
8. Decrypt the data at the server and display the result.

Python Code

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.fernet import Fernet

print("=== SSL/TLS Handshake Simulation ===")

# Server creates a public/private key pair
server_private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
server_public_key = server_private_key.public_key()

print("1. Client Hello -> Server")
print("2. Server Hello -> Client")
print("3. Server certificate/public key exchanged.")
```

```

# Client generates a session key
session_key = Fernet.generate_key()

# Protect session key using server public key
encrypted_session_key = server_public_key.encrypt(
    session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

print("4. Session key generated and securely exchanged.")

# Server recovers the session key
decrypted_session_key = server_private_key.decrypt(
    encrypted_session_key,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Secure application data transfer
cipher = Fernet(decrypted_session_key)
message = input("Enter secure data: ")

encrypted_data = cipher.encrypt(message.encode())

print("5. Encrypted application data:")
print(encrypted_data.decode())

received_data = cipher.decrypt(encrypted_data).decode()

print("6. Decrypted data at server:")
print(received_data)
print("Secure communication established successfully.")

```

Input:

```
Enter secure data: Hello Secure Server
```

Output:

```
=== SSL/TLS Handshake Simulation ===  
1. Client Hello -> Server  
2. Server Hello -> Client  
3. Server certificate/public key exchanged.  
4. Session key generated and securely exchanged.  
5. Encrypted application data:  
<encrypted ciphertext>  
6. Decrypted data at server:  
Hello Secure Server  
Secure communication established successfully.
```

Analysis

The TLS handshake establishes the cryptographic parameters and keys required before application data is transferred. Server certificates are used in real TLS to authenticate the server through a trusted certificate chain. Public-key cryptography protects the key-establishment process, while symmetric cryptography is used for efficient application-data protection after the handshake. Modern TLS versions use authenticated encryption and do not use exactly the simplified RSA key exchange shown here; this program is an educational simulation of the underlying concepts.

Result:

The program successfully demonstrates certificate/public-key exchange, session-key establishment, and secure encrypted data transfer.

QUESTION 5

Question:

Build a Python-based tool to perform Email Phishing Detection, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

Answer

Aim:

To build a simple rule-based email phishing detector using feature extraction from suspicious keywords, URL patterns, and header-related indicators, followed by threshold-based classification.

Algorithm:

1. Read the email content.
2. Convert the email to lowercase.
3. Search for suspicious keywords such as urgent, verify, password, login, and account suspended.
4. Extract URLs and check for suspicious patterns.
5. Check whether a URL directly uses an IP address.
6. Assign a phishing score based on detected features.
7. If the score reaches the threshold, classify the email as phishing; otherwise classify it as legitimate.
8. Display the score and classification.

Python Code

```
import re

def phishing_detection(email):
    score = 0
    lower = email.lower()

    suspicious_keywords = [
        "urgent", "verify", "password", "login",
        "click here", "account suspended", "confirm", "winner"
    ]

    # Keyword feature extraction
    for word in suspicious_keywords:
        if word in lower:
            score += 1
```

```

# URL feature extraction
urls = re.findall(r"https?://\S+", email)

for url in urls:
    if "@" in url or "-" in url or url.count(".") > 3:
        score += 2

# Direct IP address in URL
if re.search(r"https?://\d+\.\d+\.\d+\.\d+", lower):
    score += 3

# Threshold-based classification
if score >= 3:
    result = "PHISHING EMAIL"
else:
    result = "LEGITIMATE EMAIL"

return score, result

email = input("Enter email content: ")

score, result = phishing_detection(email)

print("\nPhishing Score:", score)
print("Classification:", result)

```

Input:

```

Enter email content: Urgent! Verify your password and login at
http://192.168.1.20/login

```

Output:

```

Phishing Score: 6
Classification: PHISHING EMAIL

```

Analysis

The detector extracts simple features from email content. Suspicious keywords increase the score, unusual URL structures add additional points, and direct IP-address URLs receive a higher penalty. The final score is compared with a threshold to classify the message.

Effectiveness:

This approach is useful for demonstrating the basic principle of feature extraction and classification and can detect obvious phishing indicators. However, it is not sufficient as

a production phishing detector because legitimate messages can contain words such as verify or login, while sophisticated phishing emails may avoid obvious indicators. A practical system should be evaluated on a labelled dataset using metrics such as accuracy, precision, recall, F1-score, and false-positive rate. Machine-learning models can also combine URL, header, domain, sender, and content features.

Result:

Thus, the program successfully demonstrates a basic phishing-detection pipeline from feature extraction to classification and highlights the need for dataset-based evaluation in real-world systems.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases